An Authoritative STM32 Battleship Server

with a Thin PySide6 Hotseat Client

Muddathir Firfirey (FRFMUD001) Adam Foflonker (FFLADA001)
EEE3095S CS Project 2026, Department of Electrical Engineering
University of Cape Town

We present a two-player Battleship game in which an STM32F051 microcontroller is the sole authority and a PySide6 graphical client on a PC only displays what the server confirms. The board holds both fleets, validates every request completely before changing any state, and never sends a player the opponent's unhit ships, so a faulty or modified client can neither cheat nor see hidden information. Client and server exchange newline-terminated ASCII lines of at most 64 bytes over a UART exposed as a USB virtual COM port; oversized, partial, corrupted and non-printable frames are rejected with specific errors or discarded after a 500 ms timeout, and every rejection leaves the game unchanged. A pytest harness drives the real board with no human input, checking both players' boards before and after every rejected request. All 159 automated tests, 53 on the board and 106 on the PC, pass on the final firmware, including every case the brief requires. Testing also exposed a UART overrun that silenced the server, which a receive ring buffer fixed.

I. Introduction

A game whose rules are enforced by the player's own computer is only as trustworthy as that computer: a modified client can make illegal moves or read information it should not have. The Serial Game Server project [1] therefore makes a microcontroller the referee. An STM32F051 board holds the only copy of the game, validates every request it receives over a serial link, and returns the authoritative result; the PC client only displays what the server confirms and forwards the player's input. Our design guarantees two properties: a rejected request never changes the game, and the server never reveals a player's unhit ships to their opponent.

We chose Battleship. The brief lists it as a strong test of a client-untrusted design, because each player's fleet is hidden information that only the server may know, and it goes beyond the Tic-Tac-Toe baseline with a ship-placement phase, a 100-cell board and five ships per player. Two players share one computer ("hotseat" play). The GUI uses PySide6 [2], the official Python binding for Qt, which the brief recommends; its threads and signals let the GUI wait for the board without freezing.

Three decisions shape the system. The protocol is strictly request/response over newline-terminated ASCII lines of at most 64 bytes, and every request that acts for a player names that player, so the server can enforce turns even though both players share one client. The firmware separates pure game logic, which also compiles and runs on a PC, from the hardware code that drives the universal asynchronous receiver-transmitter (UART), the LED and the LCD. Finally, an automated test harness drives the real board over UART with no human input, and checks after every rejected request that the game is unchanged.

Task allocation. Adam Foflonker wrote the STM32 firmware: the game logic, the UART handling, the status LED and LCD, and the build and flash tooling. Muddathir Firfirey wrote the protocol specification, the PC client library, the test harness and the GUI. We reviewed each other's work, tested the integrated system on the board together and wrote this report together.

Structure. Section II defines the message protocol. Section III presents the architecture, the server's finite-state machine (FSM), the GUI flowchart and the circuit. Section IV explains selected code. Section V shows how the server enforces the rules and why a rejected request cannot change the game. Section VI covers the handling of malformed input, and Section VII reports the test results on the board. Section VIII concludes.

II. Requirements and Message Protocol

A. Newline-terminated ASCII lines of at most 64 bytes frame the stream

The client and server communicate over USART1 of the STM32F051 (PA9 transmits, PA10 receives). The on-board ST-Link debugger exposes this UART to the PC as a virtual COM port, so a single USB cable carries power, programming and game traffic. The link runs at 115 200 baud with 8 data bits, no parity, 1 stop bit and no flow control.

Every message is one line of printable ASCII (bytes 0x20–0x7E) whose fields are separated by colons. Requests end in `\n`, and a `\r` just before it is ignored; replies end in `\r\n`. The protocol is strictly request/response: the server sends exactly one reply to every non-empty request and never transmits unprompted, so the client can pair each reply with its request without message identifiers. The client treats a missing reply after 1 s as a timeout. Table I lists the framing rules and how the server responds when a frame breaks one.

B. Each player's view is two 100-character boards, and the opponent's ships are never sent

The board is a 10×10 grid. Rows are letters `A`–`J` and columns are digits `0`–`9`, so a cell is written row then column,

TABLE I
FRAMING RULES AND THE SERVER'S RESPONSE TO EACH VIOLATION. EVERY RULE WAS CHECKED ON THE BOARD BY THE TEST HARNESS (SECTION VII).

Rule	Server behaviour
Terminator	`\n` ends a request; a preceding `\r` is ignored. Empty lines get no reply.
Maximum frame length	64 bytes. A longer frame is discarded up to the next `\n` and answered with `NACK:TOOLONG`.
Incomplete frame	Discarded without a reply once no byte has arrived for 500 ms.
Invalid character	A byte outside 0x20–0x7E makes the frame `NACK:BADCMD`.
Checksum (optional)	A `*HH` suffix is the uppercase hex XOR of the payload bytes, e.g. `CONNECT*5E`. A mismatch gives `NACK:CHECKSUM`; a checksummed request gets a checksummed reply.

e.g. `B7`. In a message, a board is a 100-character string in row-major order: cell index $= 10 \times \text{row} + \text{column}$, from `A0` (0) to `J9` (99).

`STATE:P1` returns the game as player 1 sees it: `BOARD:<turn>:<own>:<target>`. The turn field is `P1` or `P2` for the player expected to act next, or `--` once the game is over. The *own* board shows the player's waters: `.` for water, a ship letter (`C`, `B`, `R`, `S`, `D`) for an unhit ship cell, `X` for a hit on a ship and `o` for an opponent's miss. The *target* board shows only what the player has learnt about the opponent: `.` for not yet fired at, `X` for a hit and `o` for a miss. The server never sends the opponent's unhit ships, so even a modified client cannot find them.

For example, suppose player 1's Carrier lies on `A0`–`A4`, player 1 hits `A0` and player 2 misses `J9`. `STATE:P1` then returns `P1` as the turn, since it is player 1's turn again. In the own board, row `A` (the first ten characters) is `CCCCC.....` and row `J` (the last ten) is `.........o`, showing player 2's miss on `J9`. In the target board, row `A` is `X.........`, the hit on `A0`, and every other character is a dot (`.`).

C. Seven commands cover a game; the player who acts is named in each request

Table II lists every command, the states in which the server accepts it and its success reply; any other state gives an error (Table III). Commands that act for a player carry `P1` or `P2` as their first field. Both players share one client (hotseat play), so this field is how the server knows who is acting, rejects out-of-turn requests with `NACK:NOTURN`, and decides which fleet `STATE` may reveal. `CONNECT`, `NEW` and `DISCONNECT` act for neither player and have no player field.

Battleship needs fleets before play, so `NEW` starts a placing phase: player 1 places all five ships with `PLACE`, giving the ship letter, its bow cell and `H` (extends right) or `V` (extends down); then player 2 does the same. The reply to player 2's last ship is `OK:READY`, and firing starts with player 1. Each valid `MOVE` passes the turn and is answered with its result rather than a board; the client fetches the new board with `STATE`. `CONNECT` is accepted in every state and never changes the game, so a restarted client can rejoin a game in progress.

TABLE II
COMMAND SET. `<p>` IS `P1` OR `P2`, `<s>` A SHIP LETTER, `<c>` A CELL AND `<o>` `H` OR `V`. †DEPARTURE FROM THE BRIEF'S REFERENCE PROTOCOL.

Request	Accepted in	Success response
`CONNECT`	Any state	`OK`
`NEW`	`IDLE`, `GAME_OVER`	`OK:PLACE`†
`PLACE:<p>:<s>:<c>:<o>`†	`IN_PROGRESS`, placing	`OK:PLACED`, or `OK:READY` after the last ship
`MOVE:<p>:<c>`†	`IN_PROGRESS`, firing	`OK:MISS`, `OK:HIT`, `OK:SUNK:<s>`† or `WIN:<p>`
`STATE:<p>`†	`IN_PROGRESS`, `GAME_OVER`	`BOARD:<turn>:<own>:<target>`†
`RESIGN:<p>`†	`IN_PROGRESS`	`OK:GAMEOVER`
`DISCONNECT`	`IDLE`, `GAME_OVER`	`OK`

TABLE III
EVERY ERROR RESPONSE AND WHEN THE SERVER SENDS IT. NONE OF THEM CHANGES THE SERVER STATE.

Response	Sent when
`NACK:BADCMD`	Unknown command, wrong number of fields, player not `P1`/`P2`, orientation not `H`/`V`, malformed cell (e.g. `B-1`) or a non-printable byte
`NACK:TOOLONG`	Request longer than 64 bytes
`NACK:CHECKSUM`	`*HH` suffix does not match the payload
`NACK:NOSESSION`	Any command except `CONNECT` before a session is open
`NACK:NOGAME`	`PLACE`, `MOVE` or `RESIGN` with no game in progress; `STATE` in `IDLE`
`NACK:BADSTATE`	`NEW` or `DISCONNECT` during a game; `PLACE` once firing has started
`NACK:NOTREADY`	`MOVE` while ships are still being placed
`NACK:NOTURN`	`PLACE` or `MOVE` from the player who is not expected to act
`NACK:RANGE`	Cell off the board (e.g. `K3`, `A10`), or the ship would extend off it
`NACK:OCCUPIED`	`MOVE` at a cell the player has already fired at
`NACK:OVERLAP`	Ship would overlap one of the player's own ships
`NACK:DUPSHIP`	That ship has already been placed
`NACK:BADSHIP`	Ship letter is not `C`, `B`, `R`, `S` or `D`

The server never sends anything unprompted. If the board is reset, the client learns of it from its next reply, `NACK:NOSESSION` or `NACK:NOGAME`, and resynchronises by reconnecting and requesting `STATE`.

D. Thirteen specific errors, each leaving the game unchanged

Every rejected request gets one `NACK:<reason>` line, listed in Table III. A rejection never changes any server state: the board, the turn and the game phase stay exactly as they were. Section V argues why from the structure of the firmware.

When a request has several faults, the server reports the first one it finds, always checking in the same order: framing (`TOOLONG`, invalid bytes, `CHECKSUM`); syntax (`BADCMD`); session (`NOSESSION`); game state and phase (`NOGAME`, `BADSTATE`, `NOTREADY`); turn (`NOTURN`); field values (a malformed cell or orientation, then `BADSHIP`, `DUPSHIP`, `RANGE`); and finally conflicts with the board (`OVERLAP`, `OCCUPIED`). Every request therefore has exactly one correct reply; for example, `PLACE:P1:Z:K0:H` names an unknown ship at an off-board cell and gets `NACK:BADSHIP`, not `NACK:RANGE`. Each of the 13 errors is triggered by at least one test on the board (Section VII).

E. Seven departures adapt the reference protocol to hidden-information Battleship

Battleship differs from the brief's Tic-Tac-Toe reference in three ways that shape the protocol: fleets are placed before play, each player must not see the other's fleet, and both players share one client. Each departure marked † in Table II follows from one of these.

- **No `DRAW` reply.** A game ends only when a fleet is sunk or a player resigns, so it cannot end level, and the brief's draw case does not apply.
- **A placing phase with `PLACE`.** Fleets must exist before firing. Placing is a sub-phase of `IN_PROGRESS`, so the three required states remain (`NEW` replies `OK:PLACE`).
- **A player field on `PLACE`, `MOVE`, `STATE` and `RESIGN`.** With one shared client the server must know who is acting. This also makes `NACK:NOTURN` testable from a single client: the harness sends `MOVE:P2:A0` when it is player 1's turn (`test_move_out_of_turn`).
- **`BOARD` carries the turn and two boards.** Each player sees their own fleet and their shots, never the opponent's ships, and the client can redraw everything, including whose turn it is, from one reply.
- **`MOVE` replies with its result.** `OK:MISS`, `OK:HIT`, `OK:SUNK:<s>` or `WIN:<p>` tells the player what the shot did; the board follows from `STATE`.
- **Eight extra errors** (`TOOLONG`, `CHECKSUM`, `NOSESSION`, `BADSTATE`, `NOTREADY`, `OVERLAP`, `DUPSHIP`, `BADSHIP`) give every rejection a specific reason; `OCCUPIED` means "already fired at".
- **`CONNECT` in every state.** It never changes the game, so a client that was restarted or disconnected mid-game can rejoin it.

Fig. 1 shows a complete game as the client sees it, including a rejected move that leaves the turn unchanged and a move refused after the game has ended.

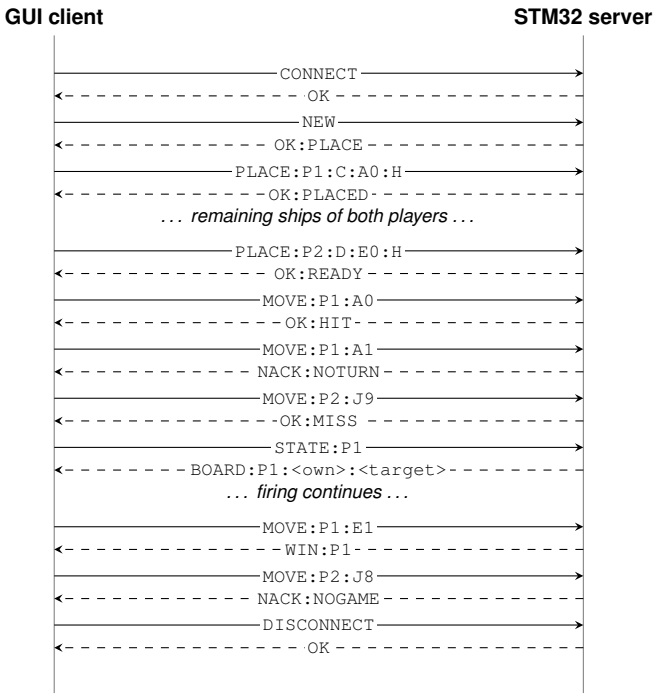


Fig. 1. A complete game recorded on the board; the elided steps are the remaining placements and shots. Requests are solid, replies dashed. The out-of-turn `MOVE:P1:A1` is rejected and leaves the turn with player 1, and moves are refused once the game is won.

III. Specification and Design

A. The server alone decides what is legal; the client only draws

The system comprises two programs joined by one UART bridge. The STM32 firmware is the referee and the PC GUI is a display. The split follows one rule: anything that decides the outcome of the game sits on the server - because a client can be modified or can crash, and the board cannot. In other words, the board is **authoritative** over the game state. On the firmware side the code is divided into two layers. The board-support layer touches hardware: the UART receive interrupt, the LCD, the status LEDs and the start-up animation. The server module (`server.c`) is pure logic with no hardware calls. It receives the link one byte at a time through `bs_server_feed(server, byte, now_ms, out)` and returns the length of the reply line, if any, so the board-support layer only moves bytes and a timestamp in and out. Inside the server the work is layered the same way a request travels: framing (64-byte limit, 500 ms partial-frame timeout, optional checksum), parsing (player, ship and cell syntax), dispatch through a command table, and one handler per command that reads and changes the game state. Read-only getters expose the state and turn so the board can show them on the LED and LCD. The client has a window that draws and a worker thread that owns the serial port, as described below.

This separation of concerns has several advantages, such as:

- Hidden information stays hidden: the `STATE` reply carries only a player's own fleet and what their shots have revealed, so even a modified client cannot learn where the opponent's ships are.
- The logic can be built and tested on a PC without the board, since it is fully-divorced from the hardware.
- The client holds no game state, so a restarted GUI recovers a game in progress by sending `CONNECT` and `STATE`.

B. Three states, with IN_PROGRESS split into placing and firing, and an ordered move check

Fig. 2 shows the server FSM. Each transition is labelled `command [guard] / response`, and the lines stacked inside a state are its internal transitions, which leave the state unchanged. The three states are `IDLE`, `IN_PROGRESS` and `GAME_OVER`; `IN_PROGRESS` has two sub-phases, placing and firing, which the firmware stores as `IN_PROGRESS_PLANNING` and `IN_PROGRESS_FIRING`.

In IDLE, NEW clears both fleets, replies OK:PLACE and enters the placing phase with P1 to act. PLACE, MOVE, RESIGN and STATE are answered with NACK:NOGAME. In IN_PROGRESS, each accepted PLACE replies OK:PLACED; when P2 places the fifth ship the reply is OK:READY and the phase changes to firing, with P1 to fire first. NEW and DISCONNECT are refused with NACK:BADSTATE, so a game cannot be thrown away by accident; RESIGN replies OK:GAMEOVER and moves to GAME_OVER. STATE is read-only and never changes the game. GAME_OVER still answers STATE, so the client can show the final board. NEW starts another game and DISCONNECT returns to IDLE; moves and placements get NACK:NOGAME. The board's reset button (NRST) is not handled by the firmware: it reboots the microcontroller, which discards the game and the session and restarts in IDLE.

A MOVE passes through the chain of checks drawn below IN_PROGRESS. They run in a fixed order, and the first failure decides the reply, so the firmware and any test double report the same error for the same bad request:

1) the frame is well formed: at most 64 bytes, printable characters only, and a matching checksum if one is sent (TOOLONG, BADCMD, CHECKSUM);
2) the syntax is valid: a known command, the right number of fields and a player of P1 or P2 (BADCMD);
3) a session is open (NOSESSION);
4) a game is in its firing phase (NOGAME in IDLE and GAME_OVER, NOTREADY while placing);
5) the sender is the player whose turn it is (NOTURN);
6) the cell parses and lies on the board (BADCMD if malformed, RANGE if off the board);
7) the cell has not been fired at before (OCCUPIED).

Every failure sends one NACK and leaves the state untouched, because the server changes nothing until all seven checks pass. Only then is the shot recorded and the turn handed to the other player. The reply is OK:MISS, OK:HIT or OK:SUNK:s for the ship that sank. If that was the opponent's last ship, the reply is WIN:Pn and the server enters GAME_OVER. There is no draw outcome: a game ends by a win or a resignation only.

The figure leaves out a few things on purpose. Checks 1 to 3 apply to every command in every state, so they are drawn once, on the MOVE chain. PLACE has its own ordered checks, collapsed into one rejection line listing the possible NACK reasons. Empty lines and partial frames that time out get no reply, so they are not transitions. CONNECT only opens a session and changes nothing else, so it is drawn as an internal line in each state. Finally, the turn passing from P1 to P2 during placing is a change of data inside IN_PROGRESS, not a transition.

C. The GUI redraws only from server replies and never waits on the serial port

The client is written in Python with PySide6 [2]. Fig. 3 shows its main loop. After the player chooses a serial port, the GUI sends CONNECT and then STATE, and every redraw that follows comes from a STATE reply: the GUI keeps no game state of its own. If the reply shows that the turn has passed to the other player, the GUI blanks both grids and covers them with a "Pass the computer to P2" screen until that player takes over, so in hotseat play one player's fleet is never left on screen for the other. Otherwise it draws the viewer's boards and enables only the grid that the phase allows: the fleet grid while ships are being placed, the shots grid while firing, and neither once the game is over. The phase itself is inferred from server data: a player whose own board shows fewer than the 17 cells of a full fleet is still placing.

A click becomes one request: PLACE, MOVE, NEW or RESIGN. A success reply (OK or WIN) is followed by STATE, and the board is redrawn from its answer. A NACK is shown in the status line as a specific message, for example "You have already fired at that cell.", and nothing is redrawn. Two replies show that the board was reset: NACK:NOSESSION makes the GUI reconnect, and NACK:NOGAME during a game makes it report that the game was lost. A timeout or a lost port is shown as an error, and Disconnect stays available, so the GUI can never be left stuck. Disconnect sends DISCONNECT, or, during a game, closes the port without it, leaving the game on the board for a later CONNECT to rejoin.

Every request waits for a reply for up to 1 s, so the GUI never calls the serial port from its own thread. A worker object that owns the port runs on a separate QThread; the window sends it requests and receives replies as Qt signals, which Qt queues between the threads, so no locks are needed and the window keeps responding while the board answers. Only one request is in flight at a time, because the controls are disabled until its reply arrives.

D. The server's physical setup

Fig. 4 presents a high-level block diagram of the server's connections to its required peripheral devices. **USART TX/RX** which are connected to GPIO pins **PA9/PA10** are mapped as virtual-COM ports by the ST-Link debugger board - which plugs directly into the PC (the client) via USB. The UART connection was configured as **115200 baud 8N1** with a common ground. The server's buttons remain completely unused. The server's LCD screen is used for displaying status messages such as which player's turn it is, and what phase (bs_state_t) the game is in. **LED D9** is the status LED, which maps a colour to a (bs_state_t) as follows:

- Green: **IDLE**
- Orange: **IN_PROGRESS_PLANNING**
- Red: **IN_PROGRESS_FIRING**
- Off: **GAME_OVER**

IV. Implementation

A. A ring buffer keeps reception running while a reply is sent

The first firmware paused reception while it transmitted each reply; bytes that arrived in that gap overran the receive register and the HAL then switched reception off for good (Section VII). The fix in Listing 1 splits the work so that

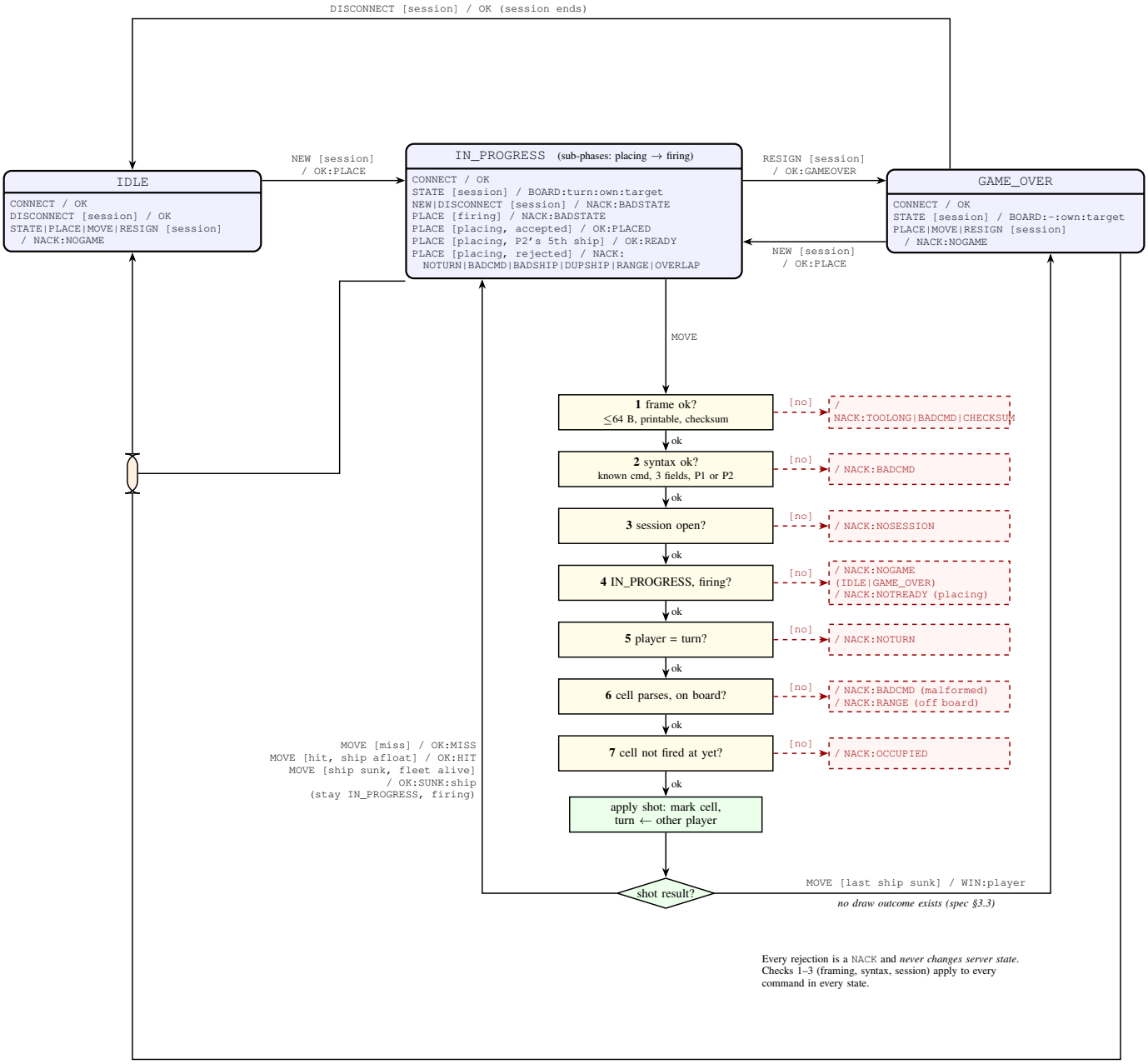


Fig. 2. Server FSM with the move-validation branch. Stacked lines inside a state are its internal transitions; the `MOVE` transition of `IN_PROGRESS` is expanded into the validation chain, each check with its `NACK` exit.

reception never stops. The USART1 interrupt handler does the minimum: it clears any error flags, reads the received byte and appends it to a 256-byte ring buffer. The main loop takes one byte per pass, feeds it to `bs_server_feed()` and, when a reply is ready, transmits it with a blocking call while the interrupt keeps filling the buffer. The obvious alternatives fail: re-arming a one-byte HAL receive after each reply still leaves a gap, and parsing or transmitting inside the interrupt would block every other interrupt for up to 18.7 ms. Only the interrupt writes the head index and only the main loop writes the tail, so the buffer needs no lock. Its size is a power of two, so the indices wrap with a mask, and it exceeds the 215 bytes that can arrive while the longest reply is sent. If it is ever full, the byte is dropped and the framing checks reject the damaged line.

```
void USART1_IRQHandler(void)
{
    if (USART1->ISR & (USART_ISR_ORE | USART_ISR_NE |
                       USART_ISR_FE | USART_ISR_PE))
        USART1->ICR = USART_ICR_ORECF | USART_ICR_NCF |
                      USART_ICR_FECF | USART_ICR_PECF;

    if (USART1->ISR & USART_ISR_RXNE) {  /* reading RDR
     clears RXNE */
        uint8_t  byte = (uint8_t)USART1->RDR;
        uint32_t next = (g_rx_head + 1u) & (RX_RING_SIZE - 1
```

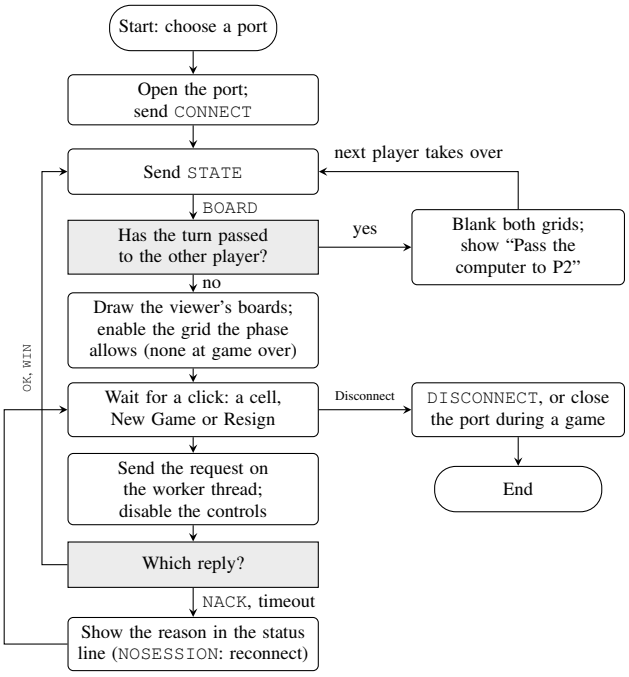


Fig. 3. GUI flowchart. Every redraw comes from a STATE reply, and each request runs on a worker thread so the window never blocks.

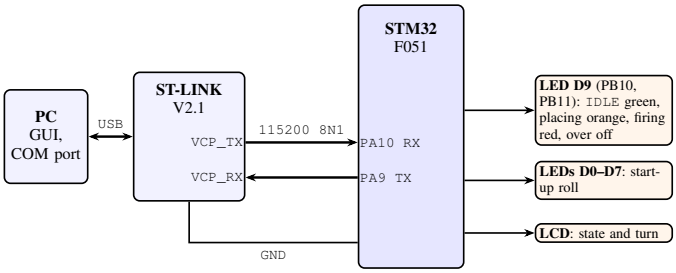


Fig. 4. Circuit: the STM32F051 talks to the PC through the ST-LINK V2.1 virtual COM port (USART1: PA9 TX, PA10 RX) with a common ground; the LEDs and LCD are on-board GPIO.

```
    u);
        if (next != g_rx_tail) {          /* full: drop the
    byte */
            g_rx_ring[g_rx_head] = byte;
            g_rx_head = next;
        }
    }
}

/* main loop: one byte per pass */
if (rx_pop(&byte)) {
    size_t len = bs_server_feed(&g_server, byte, HAL_GetTick
    (),
                                g_uart_state.tx_buf);
    if (len > 0)
        send(len);  /* blocking; the interrupt keeps
    receiving */
}
```

Listing 1. Receiving into a ring buffer in the interrupt and draining it in the main loop (main.c; trimmed).

B. A lazily checked timeout drops half-sent frames without a timer

A client that stops in the middle of a line, or a corrupted byte that hides a newline, would leave a partial frame in the receive buffer, and the next request would be glued onto it and rejected. The protocol therefore drops a partial frame after 500 ms of silence, with no reply. Instead of arming a hardware timer, the server records the time of every byte and makes the check when the next byte arrives (Listing 2). This is indistinguishable from dropping the frame at exactly 500 ms, because nothing can observe the buffer in between, and it keeps the server free of timer interrupts: the millisecond tick is passed in as now_ms, taken from HAL_GetTick [3], so the logic can be tested on a PC with a fake clock. The age is computed with unsigned subtraction, so it stays correct when the 32-bit tick wraps, which happens after about 49 days.

The same function bounds the buffer. Bytes beyond its size set an overflow flag and are discarded up to the next newline, which is then answered with NACK:TOOLONG, so an overlong line can never spill into the request after it.

```
size_t bs_server_feed(bs_server_t* server, uint8_t byte,
                      uint32_t now_ms, char* out)
{
    /* Silent for 500 ms mid-frame: drop it, no reply. */
    if (server->rx_active &&
        (uint32_t)(now_ms - server->rx_last_ms)
            >= BS_PARTIAL_TIMEOUT_MS)
        rx_reset(server);

    server->rx_last_ms = now_ms;

    if (byte != '\n')
    {
        server->rx_active = true;
        if (server->rx_len < sizeof server->rx_buf)
            server->rx_buf[server->rx_len++] = byte;
        else
            server->rx_overflow = true; /* skip to '\n' */
        return 0;
    }
    /* ... answer the complete line ... */
}
```

Listing 2. Byte-wise framing with a lazy partial-frame timeout, abridged (server.c).

C. The client discards stale input so every reply matches its request

The protocol has no message identifiers, so the client must never confuse one request's reply with another's. The risk is a reply that arrives after its request has timed out: if the client then simply read the next line, every later reply would belong to the previous request. Listing 3 shows how GameSession.request(), which the GUI and the test harness both use, prevents this: anything already buffered is discarded before a new request is sent, so the next line is the reply to this request or nothing at all. It relies on the protocol's strict request/response rule: since the server never transmits unprompted, buffered bytes can only be stale.

```
def request(self, payload: str) -> Reply:
    # Anything already buffered is a stale reply to an
    # earlier, timed-out request: drop it.
    if self.conn.in_waiting:
        self.conn.read(self.conn.in_waiting)
```

```
        self.conn.write(
            protocol.encode(payload, self.use_checksum))

    line = self.conn.readline(timeout=self.timeout)
    if line is None:
        raise TimeoutError(f"no reply to {payload!r}")
    return protocol.parse_reply(line)
```

Listing 3. Sending one request and reading its reply (`session.py`; docstring removed).

D. A worker thread keeps the GUI responsive while the board replies

`request()` blocks for up to 1 s, so calling it from a button's click handler would freeze the window: no repainting, unresponsive controls. Running it on a plain Python thread is no better, because Qt widgets may only be updated from the GUI thread. Listing 4 shows the approach used: a `SerialWorker` that owns the port is moved to a `QThread`, and the window talks to it only through signals. Qt queues a signal emitted in one thread to an object living in another, so `run()` executes on the worker thread and `on_finished()` back on the GUI thread, without locks. Each request is a function of the session, so the window decides what to send and the worker only executes it; the tag tells the window what a reply belongs to, e.g. `fire:B7`. Setting `busy` disables the controls until the reply arrives, so only one request is ever in flight.

```
# main_window.py: the worker lives on its own thread
self.worker_thread = QThread(self)
self.worker.moveToThread(self.worker_thread)
self.worker_thread.start()
self.run_requested.connect(self.worker.run)
self.worker.finished.connect(self.on_finished)

def send(self, tag: str, request: Request):
    self.busy = True        # one request in flight
    self.update_controls()  # disables the controls
    self.run_requested.emit(tag, request)

# worker.py: runs on the worker thread
@Slot(str, object)
def run(self, tag: str, request: Request):
    try:
        self.finished.emit(tag, request(self._session))
    except _REQUEST_ERRORS as e:
        self.failed.emit(tag, str(e))
```

Listing 4. The worker thread and its signals (`main_window.py` and `worker.py`; trimmed).

V. Game Logic and Server Authority

A. A move passes seven ordered checks, and the first failure names the error

Every request is judged in the fixed order of Fig. 2, and the reply names the first check that fails. Illegal moves get `RANGE`, `OCCUPIED`, `OVERLAP`, `DUPSHIP` or `BADSHIP`; out-of-turn requests `NOTURN`; out-of-state requests `NOGAME`, `NOTREADY` or `BADSTATE`. Checks 1 to 3 (framing, syntax, session) run in `bs_server_feed`, `process_frame` and `dispatch_command` before any handler is called. Checks 4 to 7 open each handler, as in `command_move` (abridged):

```
if (state is IDLE or GAME_OVER) return NOGAME;
if (state is PLANNING)          return NOTREADY;
if (player != server->turn)     return NOTURN;
cr = parse_cell(f[2], &idx);
if (cr != CELL_OK)    return BADCMD or RANGE;
if (target->shot[idx])          return OCCUPIED;
target->shot[idx] = 1;      /* first write */
```

`command_place` uses the same order, ending with the board edge (`RANGE`) and overlap (`OVERLAP`).

B. Every check runs before the first write, so a rejection changes nothing

The *game state* is `state`, `turn`, `session` and the two fleets. The receive buffer is excluded: it changes with every byte, valid or not. The claim is that a request answered with a `NACK` leaves the game state unchanged.

1) Every error is `return nack(out, reason)`, which writes only to the reply buffer and leaves the function.
2) Framing errors return before the request is dispatched, and `dispatch_command` rejects bad syntax and a missing session before calling a handler. Up to there nothing in the game is written: parsing works on a local copy of the line, and the `parse_*` helpers write only locals.
3) In every handler the writes follow the last `NACK`. `command_new`, `command_disconnect` and `command_resign` check once, then write. `command_place` writes only in its commit block, after the read-only overlap loop. `command_move` writes after `OCCUPIED`. `command_state` never writes.
4) So a path that ends in a `NACK` never reaches a write, and nothing after the first write can fail, so no handler stops half-way.

The harness checks this on the board: every test of a rejected request, such as `test_occupied_cell_rejected` and those in the placement, turn and framing groups, compares both players' boards before and after the request.

C. One turn counter is written in four places, all after validation

The turn is one field, `turn` (0 for `P1`, 1 for `P2`), and the player parser only yields 0 or 1. It is written in `game_reset` (`P1`); in `command_place` (to `P2` after `P1`'s fifth ship, back to `P1` when `P2`'s fifth starts firing); and in `command_move` (to the other player after every valid shot, hit or miss). `command_place` and `command_move` reject a mismatch with `NOTURN`, after the state check and before any look at the board. `STATE` and `RESIGN` are not turn-gated: one only reads, and a player may forfeit at any time.

D. A game ends at the first fleet destroyed, and a draw cannot occur

Each fleet counts the unhit cells of every ship. In `command_move` the shot is recorded, then an empty cell gives `OK:MISS`, a ship with cells left gives `OK:HIT`, and a sunk ship gives `WIN:P`n if all five ships are sunk (the state becomes `GAME_OVER` before the reply) and `OK:SUNK:`s otherwise. Testing for the win first makes the last sinking report `WIN` once. Afterwards `MOVE`, `PLACE` and `RESIGN` get `NOGAME` as their first check.

A draw cannot happen, so none is detected. A shot changes only the opponent's fleet and the game stops at the first fleet destroyed, so both cannot be destroyed. A game also cannot stall: each accepted shot marks a new cell, so at most 100 shots reach a board, and they include all 17 ship cells ($5 + 4 + 3 + 3 + 2$). The protocol therefore has no `DRAW` reply.

E. The client holds no rules, so a faulty or modified client cannot cheat

The client never decides whether a request is legal. It does not check placements for overlap or the board's edge, does not track whose turn it is, does not remember which cells were fired at and does not detect a win. It keeps no game state of its own either: everything it draws comes from the last `STATE` reply, and every rejection it shows is the server's `NACK`. The client uses game knowledge in one place only, to choose what to display: it infers the phase from the server's board (fewer than 17 ship cells means the player is still placing) and disables controls the server would reject. These are conveniences; the server still checks every request it receives.

If the client enforced rules instead, two copies of each rule would have to agree, and they drift apart. Our own firmware briefly reported errors in a different order from the specification (Section VII), exactly the kind of disagreement duplicated rules cause. Worse, a client that skipped its own checks, through a bug or deliberately, could place overlapping ships, fire twice at a cell or move out of turn. The harness shows that this is impossible: it sends requests that the GUI never would, such as out-of-turn moves, overlapping ships, malformed frames and moves after the game has ended, and the server rejects each one without changing the game. Because the server never sends the opponent's unhit ships, no client, however modified, can reveal them.

VI. Protocol Robustness and Integrity

The server treats malformed input as just another request: each bad frame costs one specific reply or a silent discard, never a change of state, and the next newline always resynchronises the stream. Table IV lists each kind of bad input, how the server handles it and the board test that checks it.

A. A partial frame is dropped after 500 ms of silence

If no byte arrives for 500 ms while a frame is incomplete, the server discards it without a reply. The check runs when the next byte arrives: the server compares the millisecond tick with the time of the previous byte using unsigned subtraction, so the rule still holds when the 32-bit tick wraps around. A full 64-byte frame takes 5.6 ms to transmit at 115 200 baud and the client writes each request in one call, so a 500 ms gap can only mean that the sender stopped mid-frame, for example a client that crashed. The client also sends a bare `\n` before `CONNECT`, which flushes any fragment left on the server. On the board, a frame split by a 100 ms pause was joined and executed, and a fragment followed by a 700 ms pause was discarded, so the next request was answered normally.

TABLE IV
Bad input, the server's handling, and the test on the board that checks it.

Bad input	Handling and response	Test
Partial frame	Discarded silently once 500 ms pass without a byte	`test_partial_frame_completed_within_timeout`, `test_partial_frame_discarded_after_timeout`
Oversized frame	Over 64 bytes: the rest is discarded up to the next `\n`, then `NACK:TOOLONG`	`test_frame_length_limit`
Garbage bytes	Any byte outside 0x20–0x7E: `NACK:BADCMD`; the next `\n` resynchronises	`test_bad_frame_rejected`, `test_server_survives_garbage_burst`
Wrong case or unknown command	Exact, case-sensitive match against the command table: `NACK:BADCMD`	`test_bad_frame_rejected`
Corrupted bytes (checksum)	XOR mismatch: `NACK:CHECKSUM`; the request is not executed	`test_bad_checksum_rejected`
UART errors (overrun, framing)	The interrupt handler clears the error flags and keeps receiving into a 256-byte ring buffer	`test_back_to_back_requests`, `test_server_survives_garbage_burst`

B. Garbage and oversized input cost one reply and resynchronise at the next newline

Every `\n` ends a frame, whatever came before it, so after any amount of garbage the next well-formed line is processed normally. A frame longer than 64 bytes sets an overflow flag: the server stores nothing more until the next `\n` and then replies `NACK:TOOLONG` once, so an oversized frame cannot overrun the receive buffer. On the board, 64 bytes were accepted as a frame and 65 were rejected, and after a 1 KB burst containing every byte value four times the board still answered and both players' boards were unchanged. The optional checksum detects any single-bit error in a frame, but not every multi-byte error, since two corruptions of the same bit cancel in an XOR.

C. The server never waits for input and bounds every step

The main loop never blocks on input: each pass takes at most one byte from the ring buffer, so the LED and LCD updates run even during a burst. The work done for a complete frame is bounded: at most 64 bytes are parsed into at most five fields, and the longest reply is built from the two 100-cell boards. The only blocking call is the transmission of a reply; the longest reply is 215 bytes on the wire and takes 18.7 ms at 115 200 baud. Reception continues in the interrupt handler meanwhile, and the 256-byte ring buffer holds more than the 215 bytes that can arrive in that time. The one way the server can stop is a failure reported by the HAL, after which the error handler halts until the board is reset (Section VIII). The full test suite of 159 tests ran on the board without the server ever stopping.

VII. Validation

A. A pytest harness drives the real board over UART with no human input

The test harness uses pytest [4] and pySerial [5]. It talks to the board through the same request/reply session class as

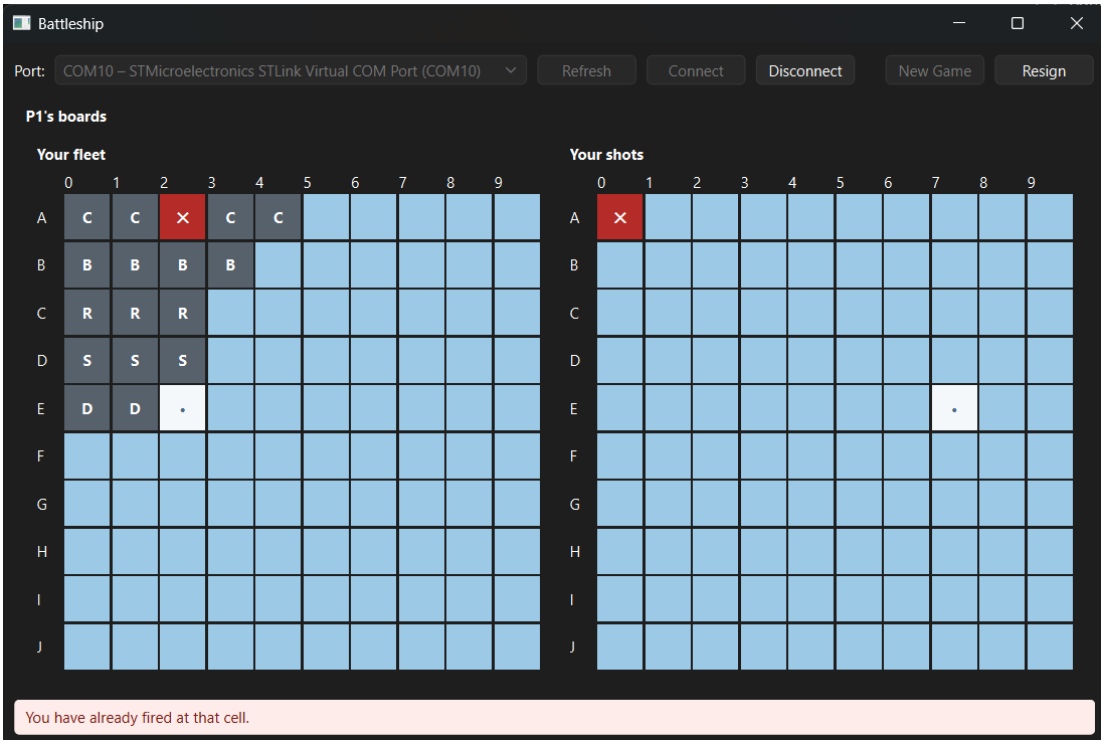


Fig. 5. The GUI connected to the board during player 1's turn, after player 1 clicked a cell they had already fired at. The server's `NACK:OCCUPIED` is shown as a specific message in the error style, and both boards are unchanged because the GUI redraws only from server replies.

the GUI, so it exercises exactly the path a player's actions take. Running `uv run pytest --port COM10` executes 159 tests: 53 *board tests* that drive the STM32 over UART, and 106 PC-side tests of the protocol parser, the session, the background worker and the GUI widgets. The PC-side tests use a fake serial port and draw the GUI off-screen, so they also run in continuous integration on every pull request; without `--port`, the board tests are skipped. Every server result in this section comes from the real board; no simulator was used.

Each board test starts from a known state: a fixture sends `CONNECT`, asks for `STATE` and resigns any game left over from an earlier test, so the tests are independent of their order. Both players use a fixed fleet layout (one ship per row, `A`–`E`), so every hit, sink and win can be scripted exactly. Every test of a rejected request records both players' boards before and after the request and asserts that they are identical, which checks the guarantee that a rejection never changes the game. The framing tests bypass the session and write raw bytes, including oversized, partial, corrupted and non-printable frames.

B. All 159 tests pass on the board, covering every required case

Table V maps each test the brief requires to the harness test that checks it. Tests that inspect the board compare all 100 cells of both players' views, not just the cell that should change, so an unexpected change anywhere fails the test. The scripted win checks all 17 replies of a game, so it also confirms that each ship is reported sunk on exactly its last cell.

TABLE V
THE BRIEF'S REQUIRED TESTS AND THEIR RESULTS ON THE BOARD.

Requirement (brief, p. 4)	Test name	Result
CONNECT returns OK; NEW returns an empty board	`test_connect_returns_ok`, `test_new_returns_empty_board`	Pass
A legal move changes the board exactly as expected	`test_legal_move_changes_one_cell` (hit and miss)	Pass
Occupied cell returns NACK:OCCUPIED, board unchanged	`test_occupied_cell_rejected`	Pass
Out-of-range move returns NACK:RANGE	`test_out_of_range_move_rejected` (`K3`, `A10`)	Pass
Move after game over returns NACK:NOGAME	`test_move_after_game_over_rejected`	Pass
Scripted win gives the correct result	`test_scripted_win`	Pass
Scripted draw gives the correct result	Not applicable: Battleship cannot end in a draw (Section II)	n/a
AI never makes an illegal move	Not applicable: no AI opponent	n/a

Beyond the required cases, 14 placement tests cover every way a `PLACE` can be rejected, 13 tests cover turn order, out-of-state requests, resigning, disconnecting and hidden information, and 17 framing tests send malformed, oversized, partial and corrupted frames. Together they trigger each of the 13 error responses at least once. The final run on the board, after the firmware fixes described below, reported:

```
battleship target: board on COM10
159 passed in 21.72s
```

Fig. 5 shows the GUI connected to the board during a game, immediately after the server rejected a shot.

C. Testing on the board exposed a UART overrun that silenced the server

The most serious fault appeared only on the hardware. Sending two requests in one write, or a burst of garbage, made the board answer once and then ignore every request until it was reset (`test_back_to_back_requests` and `test_server_survives_garbage_burst`). Halting the processor through the ST-Link with OpenOCD showed it still running the main loop, so the firmware had not crashed. Reading USART1's registers [6] explained the silence: the overrun flag was set (`ISR.ORE` $= 1$) and the receive interrupt was disabled (`CR1.RXNEIE` $= 0$). The firmware paused reception while it transmitted each reply, which takes up to about 18.7 ms at 115 200 baud. Bytes that arrived in that gap overran the one-byte receive register, the HAL treats an overrun as a blocking error and stops reception [3], and nothing re-enabled it. The fix keeps reception running permanently: the USART1 interrupt handler clears any error flags and appends every byte to a 256-byte ring buffer, and the main loop feeds the buffered bytes to the server and sends the replies. After the fix both tests pass, and the board keeps answering after a 1 KB garbage burst.

The placement tests found a second, smaller fault: when a `PLACE` request had two faults, the firmware reported `RANGE` before `BADSHIP` or `DUPSHIP`, against the agreed order of checks. `test_placement_check_order` was marked as an expected failure until the firmware was changed to check the ship before the cell's range.

Before the harness existed, compiling the platform-independent game logic on a PC and running scripted games against it found three logic errors during code review: `MOVE` was accepted only during placement, `PLACE` read the ship letter from the cell field, and the sunk reply lacked a separator (`OK:SUNKD`). All were fixed before the firmware was integrated with the UART.

VIII. Conclusion

We built a Battleship game in which an STM32F051 board is the only authority: it holds both fleets, validates every request before changing any state and never sends a player the opponent's unhit ships, while a PySide6 GUI lets two players share one computer. The system meets the brief's requirements: the three-state server rejects illegal, out-of-turn and out-of-state requests with 13 specific errors; the protocol frames, bounds and checksums its messages and recovers from partial and garbage input; and the GUI connects, starts games, submits moves, shows each rejection, announces the winner, disables moves after the game and disconnects cleanly. All 159 automated tests, including every required case, pass on the board (Section VII).

Usefulness. The same pattern suits any embedded device controlled by software it cannot trust, such as a laboratory instrument or an industrial controller driven from a PC: keep the state on the device, check every request completely before acting on it, answer each request with one specific reply, and test the device itself through its real interface.

Limitations. Both players share one client, and the player field in a request is not authenticated, so the server cannot tell two people apart; this is acceptable for hotseat play but not for separate clients. The firmware does not use the board's user button: the only reset is the hardware reset, which reboots the microcontroller and discards the session as well as the game. The server never sends unprompted messages, so the GUI learns of a reset only from its next reply, and a request sent during the 1.1 s startup animation goes unanswered until the client retries. If a HAL call fails, the firmware's error handler stops the server until the board is reset. Finally, a ship placed in the wrong position cannot be moved; the players must resign and start again.

Improvements. Two-client play, a stretch feature from the brief, would need a second UART so that each player's identity comes from the port their request arrives on rather than from a field they could forge. Undoing a ship placement, and later a move, would make the game friendlier, and an AI opponent would allow single-player games.

IX. Video

The demo video is available at DemoVideo.mp4. This video showcases the full behaviour of the game and server, and covers all possible avenues and cases the game could end up in. The following scenarios have are demonstrated in the video:

- Server startup sequence.
- Resetting the server (NRST).
- Client connecting to the server.
- Playing through a full game where a player wins.
- Disconnecting midway through a game.
- Reconnecting after disconnecting from a game.
- Resigning from a game (forfeit).
- Resetting the server during an active game.

X. Code

The source code is at DnA-IntRicate/battleship. This repository contains all the source code and the files required to run the project. Instructions on how to run the project can be found in 'README.md'.

References

[1] Department of Electrical Engineering, University of Cape Town, "EEE3095S CS project 2026: Serial game server," Course project brief, 2026.

[2] The Qt Company, "Qt for Python (PySide6) documentation," https://doc.qt.io/qtforpython-6/, version 6.12.

[3] *UM1785: Description of STM32F0 HAL and Low-Layer Drivers*, STMicroelectronics.

[4] pytest-dev team, "pytest documentation," https://docs.pytest.org/.

[5] C. Liechti, "pySerial documentation," https://pyserial.readthedocs.io/.

[6] *RM0091: STM32F0x1/STM32F0x2/STM32F0x8 Advanced Arm-based 32-bit MCUs, Reference Manual*, STMicroelectronics.